

Task 1: Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

```

shreyas@Grigori:~$ nano counter_race.c
shreyas@Grigori:~$ gcc counter_race.c -o counter_race -pthread
shreyas@Grigori:~$ ./counter_race
Expected counter value: 4000000
Actual counter value: 1056103
shreyas@Grigori:~$ ./counter_race
Expected counter value: 4000000
Actual counter value: 1345888
shreyas@Grigori:~$ ./counter_race
Expected counter value: 4000000
Actual counter value: 1336863
shreyas@Grigori:~$ ./counter_race
Expected counter value: 4000000
Actual counter value: 1238002
shreyas@Grigori:~$ ./counter_race
Expected counter value: 4000000
Actual counter value: 1486852
shreyas@Grigori:~$ ./counter_race
Expected counter value: 4000000
Actual counter value: 1274813
shreyas@Grigori:~$ ./counter_race
Expected counter value: 4000000
Actual counter value: 1267950
shreyas@Grigori:~$ |

```

```

GNU nano 7.2 counter_race.c
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 1000000

int counter = 0;

void *increment_counter(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++)
    {
        counter++;
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL, increment_counter, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter value: %d\n",
        NUM_THREADS * ITERATIONS);

    printf("Actual counter value: %d\n", counter);
}

```

Task 2: Modify the previous program using mutex locks (pthread_mutex_lock, pthread_mutex_unlock) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

```
shreyas@Grigori:~$ nano counter_mutex.c
shreyas@Grigori:~$ gcc counter_mutex.c -o counter_mutex -pthread
shreyas@Grigori:~$ ./counter_mutex
Expected counter value: 40000000
Actual counter value: 40000000
shreyas@Grigori:~$ ./counter_mutex
Expected counter value: 40000000
Actual counter value: 40000000
shreyas@Grigori:~$ ./counter_mutex
Expected counter value: 40000000
Actual counter value: 40000000
shreyas@Grigori:~$ ./counter_mutex
Expected counter value: 40000000
Actual counter value: 40000000
shreyas@Grigori:~$ ./counter_mutex
Expected counter value: 40000000
Actual counter value: 40000000
shreyas@Grigori:~$ |
```

```

GNU nano 7.2
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 1000000

int counter = 0;

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *increment_counter(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++)
    {
        pthread_mutex_lock(&mutex);

        counter++;

        pthread_mutex_unlock(&mutex);
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL, increment_counter, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter value: %d\n",
        NUM_THREADS * ITERATIONS);

    printf("Actual counter value: %d\n", counter);

    pthread_mutex_destroy(&mutex);

    return 0;
}

```